

# Lecture Notes on Functions, Recursion, and Memory Management in C

Mohammad Maksood Akhter, Ph.D.

Assistant Professor

Department of Computer Science and Engineering  
ITER, Siksha 'O' Anusandhan (Deemed to be University)

## Abstract

These lecture notes present a comprehensive study of **functions and recursion** in the C programming language, emphasizing modular design, code reusability, and execution flow. The discussion extends to **memory segmentation, storage classes, and variable scope**, offering a detailed understanding of how C manages data, activation records, and memory allocation. Each section includes illustrative examples, syntax explanations, and conceptual diagrams to build a strong foundation in structured programming and low-level execution behavior.

## I. FUNCTIONS AND RECURSION IN C

Functions in C are fundamental building blocks that enable modular programming, code reuse, and logical structure. They divide a large program into smaller, manageable subprograms.

### A. 1. Introduction to Functions

**Definition:** A function is a block of code that performs a specific task. Functions promote **code reuse** and improve readability.

#### Difference between Loops and Functions:

- A **loop** is used for *code utilization* — repeating instructions.
- A **function** is used for *code iteration and reuse*.

*Function Prototype, Definition, and Call: Syntax:*

```
return_type function_name(parameter_type1, parameter_type2, ...);
```

**Example:**

```
1  #include <stdio.h>
2
3  int add(int, int); // Function Prototype
4
5  int main() {
6      int a;
7      a = add(10, 20); // Function Call
8      printf("%d", a);
9      return 0;
10 }
11
12 int add(int x, int y) { // Function Definition
13     int z;
14     z = x + y;
15     return z;
16 }
```

**Output:**

If a function is to be reused in multiple programs, it should be declared in a **header file** and included in each program using `#include`. This promotes code reusability and modular programming.

```

1      // add.h   Function prototype stored in header file
2      int add(int, int);
3
4      // add.c   Function definition
5      int add(int a, int b) {
6          return a + b;
7      }
8
9      // main.c   Reusing the function
10     #include "add.h"
11     #include <stdio.h>
12
13     int main() {
14         int result = add(10, 20);
15         printf("Sum = %d\n", result);
16         return 0;
17     }

```

## II. 2. RECURSION

**Definition:** A function calling itself directly or indirectly is called a **recursive function**.

A recursive function must have a **termination condition**, otherwise it will result in infinite recursion and eventually a stack overflow.

**Example:**

```

1      #include <stdio.h>
2
3      void mu(int l) {
4          printf("%d\n", l);
5          if (l <= 2)
6              mu(l + 1);
7          printf("%d\n", l);
8      }
9
10     int main() {
11         mu(1);
12         return 0;
13     }

```

**Output:**

```

1
2
3
3
2
1

```

Every recursive call creates a new **activation record** (stack frame) in memory.

mu(3)

mu(2)

mu(1)

Each call is pushed on stack. When a function returns, its activation record is popped from the stack.

### A. 3. Types of Recursion

#### Tail Recursion

A recursive function is said to be a **tail recursive function** if the recursive call is the last statement in the function and nothing is left to execute after the recursive call returns. It allows the compiler to optimize the recursive call into an iterative loop (known as tail-call optimization).

```

1      int fact(int n) {
2          if (n == 0)
3              return 1;
4          else
5              return n * fact(n - 1); // last statement is recursive call
6      }

```

**Output:** fact(5) = 120

#### Non-Tail Recursion

In **non-tail recursion**, the recursive call is not the last statement in the function. After the recursive call returns, additional computations or statements are executed. This type of recursion cannot be easily optimized because the function has pending work after each call returns.

```

1      void A(int n) {
2          if (n <= 1) return;
3          A(n - 2);
4          printf("%d ", n); // statement after recursive call
5          A(n - 1);
6      }

```

**Output:** 3 2 0 1 5 2 0 4 3 2 0 1 2 3

### Indirect Recursion

When two or more functions call each other in a circular manner, it is known as **indirect recursion**. Each function participates in the recursion chain until a base condition is met. This form of recursion is common in systems with mutually dependent operations.

```

1      void B(int n);
2
3      void A(int n) {
4          if (n <= 1) return;
5          B(n - 2);
6          printf("%d ", n);
7      }
8
9      void B(int n) {
10         if (n <= 1) return;
11         A(n - 1);
12         printf("%d ", n);
13     }
14
15     int main() {
16         A(5);
17     }

```

**Output:** 2 3 4 5

### Nested Recursion

A function is said to follow **nested recursion** when a recursive function passes itself as an argument in another recursive call. This results in multiple layers of recursion, where each inner call must complete before the outer call can continue.

```

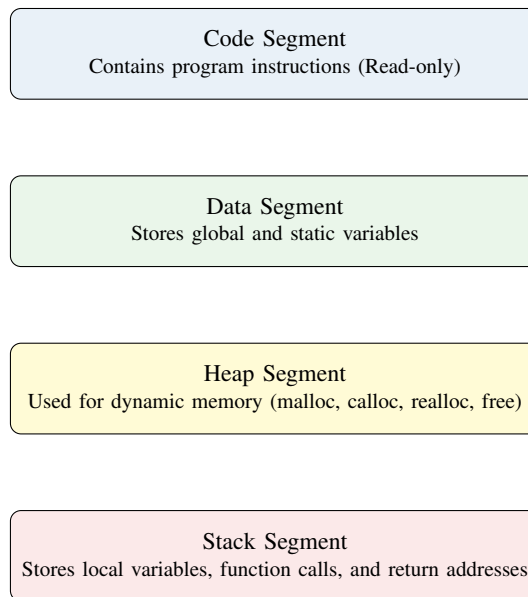
1      int A(int m, int n) {
2          if (m == 0)
3              return n + 1;
4          else if (n == 0)
5              return A(m - 1, 1);
6          else
7              return A(m - 1, A(m, n - 1));
8      }
9
10     int main() {
11         printf("%d", A(1, 5));
12         return 0;
13     }

```

**Output:** 7

## III. 4. MEMORY SEGMENTS IN C

In C programming, every process is divided into multiple memory segments. Each segment serves a specific purpose for program execution, variable storage, and function management. The four major memory segments are illustrated below.



### 1. Code Segment (Text Segment)

The code segment, also known as the **text segment**, contains the compiled machine code (instructions) of the program. It is typically marked as **read-only** to prevent accidental modification of instructions during runtime.

**Example:** When the program executes, statements like `printf()` and `return 0;` are fetched from this segment.

### 2. Data Segment

This section holds all **global variables** and **static variables**. It is divided into two parts:

- **Initialized data segment:** Stores global/static variables with explicit initialization (e.g., `int x = 10;`).
- **Uninitialized data segment (BSS):** Stores global/static variables without initialization (e.g., `static int count;`).

**Example:**

```

1  int a = 10;           // Stored in initialized data segment
2  static int b;        // Stored in uninitialized (BSS) segment

```

### 3. Heap Segment

The heap is used for **dynamic memory allocation**, which allows memory to be reserved at runtime using functions such as `malloc()`, `calloc()`, or `realloc()`. Memory from the heap must be explicitly released using `free()` to avoid memory leaks.

**Example:**

```

1  int *ptr = (int*) malloc(5 * sizeof(int)); // Allocated in heap
2  // use memory
3  free(ptr); // Released back to heap

```

#### 4. Stack Segment

The stack segment is used to store:

- Local variables of functions.
- Function parameters and return addresses.
- Activation records for recursive calls.

Memory in the stack is automatically managed — allocated when a function is called and released when it returns.

**Example:**

```

1      void func() {
2          int x = 10;  // Stored in stack
3      }
4      int main() {
5          func();      // Stack frame created for func()
6          return 0;
7      }

```

#### Summary of Memory Segments:

- **Code Segment** — Contains executable instructions (read-only).
- **Data Segment** — Stores global and static variables (both initialized and uninitialized).
- **Heap Segment** — Used for dynamic memory allocation via pointers.
- **Stack Segment** — Holds local variables, function parameters, and activation records.

### IV. 5. STORAGE CLASSES IN C

In C, every variable is defined not only by its **data type** but also by its **storage class**. Storage classes determine four key properties of a variable:

- **Data Type** — Specifies the kind of data (e.g., int, float, char).
- **Scope** — The region or block of code where the variable is accessible.
- **Lifetime** — The duration for which the variable exists in memory.
- **Default Value** — The initial value if not explicitly initialized.

There are four primary storage classes in C: auto, register, static, and extern.

#### (a) Automatic Variable (auto)

Automatic variables are local variables that are created automatically when a function is called and destroyed when the function exits. They are stored in the **stack segment** of memory and have **block scope** and **local lifetime**. If not initialized, they contain **garbage values**.

**Syntax:**

```

1      auto data_type variable_name;

```

**Example:**

```

1      #include <stdio.h>
2      void text() {
3          auto int i = 10;  // auto variable, local to this function
4          printf("%d ", i);
5          i++;
6      }
7
8      int main() {
9          text();
10         text();
11         text();
12         return 0;
13     }

```

**Output:** 10 10 10

Each time the function is called, a new copy of the auto variable is created and initialized again.

**(b) Register Variable (`register`)**

The `register` storage class is used to request that the compiler store the variable in a CPU register instead of the main memory, enabling faster access. This is particularly useful for loop counters or frequently accessed variables. If no register is available, the variable behaves like an `auto` variable.

**Syntax:**

```
1 register data_type variable_name;
```

**Example:**

```
1 #include <stdio.h>
2 int main() {
3     register int count;
4     for (count = 1; count <= 5; count++) {
5         printf("%d ", count);
6     }
7     return 0;
8 }
```

**Output:** 1 2 3 4 5

**Note:** The keyword `register` is a *request*, not a command — the compiler may or may not honor it depending on system architecture and register availability. Register variables cannot have their addresses taken using the `&` operator.

**(c) Static Variable (`static`)**

A `static` variable preserves its value between multiple function calls. It is initialized only once and resides in the **data segment** of memory (not the stack). If not explicitly initialized, it defaults to 0. Static variables can be either:

- **Local static** — defined inside a function but retain value between calls.
- **Global static** — visible only within the same source file (provides internal linkage).

**Syntax:**

```
1 static data_type variable_name;
```

**Example:**

```
1 #include <stdio.h>
2 void text() {
3     static int i = 10; // retains its value between calls
4     printf("%d ", i);
5     i++;
6 }
7
8 int main() {
9     text();
10    text();
11    text();
12    return 0;
13 }
```

**Output:** 10 11 12

A static variable is created only once during the entire execution of the program and maintains its previous value across function calls.

**(d) External Variable (extern):**

The `extern` storage class is used to declare a global variable that is defined in another file or scope. It allows sharing of variables across multiple files within the same program. Extern variables have **global scope**, **program lifetime**, and default value 0.

**Syntax:**

```
extern data_type variable_name;
```

**Example:**

```
// file1.c
#include <stdio.h>
int count = 5; // global variable

// file2.c
extern int count; // declaration (defined elsewhere)
int main() {
    printf("%d", count);
    return 0;
}
```

**Output:** 5

The `extern` keyword is primarily used in large projects to share global data between multiple source files, promoting modular programming.

## V. 6. SCOPE AND VARIABLE BINDING

Every variable in C has a specific **scope** and **lifetime**:

- **Scope:** The region or block of code where a variable can be accessed.
- **Lifetime:** The duration for which the variable exists in memory during program execution.

Understanding these properties is crucial for memory management, modular programming, and debugging variable-related errors.

A **free variable** is a variable that is used inside a function but not declared within it. Its value is resolved according to the language's scoping rule — either **static (lexical)** or **dynamic**.

### (a) Static Scoping (Lexical Scoping)

**Definition:** In **static scoping**, also called **lexical scoping**, the binding of a variable reference to its declaration is determined at **compile-time** based on the physical structure of the program code. The compiler resolves variable names by searching outward through the nested blocks of code where the variable is defined.

**Example:**

```
#include <stdio.h>

int x = 10; // Global variable

void func() {
    printf("%d\n", x); // Refers to global x
}

int main() {
    int x = 20; // Local variable in main
    func(); // Output -> 10 (global x)
    return 0;
}
```

**Output:** 10

In **static (lexical) scoping**, variable resolution depends on the code structure — not on the order of function calls. C always uses **static scoping**.

### (b) Dynamic Scoping (Conceptual Example)

**Definition:** In **dynamic scoping**, the binding of variables is determined at **runtime**, based on the calling sequence of functions. When a variable is referenced, the program searches for its most recent active declaration in the call stack. Although C does *not* support dynamic scoping, understanding it helps in comparing different language behaviors.

**Conceptual Example:**

```

1  int x = 10; // Global variable
2
3  void func() {
4      printf("%d\n", x); // Which x? depends on call context
5  }
6
7  void main_func() {
8      int x = 20; // Local to main_func
9      func(); // Under dynamic scoping -> prints 20
10 }
11
12 int main() {
13     main_func();
14     return 0;
15 }

```

**Output (if dynamic scoping were used):** 20

Under **dynamic scoping**, variable lookup happens at runtime — the interpreter searches the call stack for the most recent variable declaration. This approach is used in some scripting or interpreted languages (e.g., Bash, early versions of Lisp), but not in C.

### (c) Summary of Scoping Rules

#### Comparison: Static vs Dynamic Scoping

| Aspect                 | Static Scoping                   | Dynamic Scoping                   |
|------------------------|----------------------------------|-----------------------------------|
| Variable Binding Time  | Compile-time                     | Runtime                           |
| Variable Resolution    | Based on program structure       | Based on function call chain      |
| Performance            | Faster (compile-time resolution) | Slower (runtime lookup)           |
| Used In                | C, C++, Java, Python             | Lisp (old), Bash, Perl (optional) |
| Example Output (above) | 10                               | 20                                |

TABLE I  
STORAGE CLASS SPECIFIERS IN C

| Type            | Scope       | Lifetime | Default Values        | Storage  | Memory Segment        |
|-----------------|-------------|----------|-----------------------|----------|-----------------------|
| <i>auto</i>     | body        | body     | Garbage (undefined)   | memory   | Stack                 |
| <i>register</i> | body        | body     | Garbage (undefined)   | register | CPU Registers / Stack |
| <i>static</i>   | body/global | program  | 0                     | memory   | Data segment          |
| <i>extern</i>   | global      | program  | Depends on definition | memory   | Data segment          |

#### A. 7. Summary of Key Points

- Functions modularize code for better reuse.
- Recursion allows a function to solve problems by calling itself.
- Memory in C is divided into Code, Data, Stack, and Heap segments.
- Storage classes define scope and lifetime.
- Static scoping (used in C) resolves names based on program structure.